

Simulador de planificación de procesos

Por:

Johan Esteban Galeano Carvajal

Programa:

Tecnología en Desarrollo de Software

Curso:

Sistemas Operativos

Docente:

Yesid Montenegro García

Institución Universitaria Pascual Bravo

Medellín, Colombia

2026

SIGET: Reporte técnico y decisiones de diseño

1. Objetivos y decisiones de diseño

La idea central del proyecto fue simular la captura y el procesamiento de datos de tráfico urbano en tiempo real, cuidando que no aparecieran condiciones de carrera, que no se perdiera información y que la memoria no se desbordara.

Para representar las limitaciones de memoria de una infraestructura IoT real, se usó una cola FIFO compartida (`shared_buffer`, implementada con `collections.deque`) de tamaño fijo, con capacidad para 5 elementos.

Uno de los puntos que más cuidado requirió fue el cierre limpio del sistema. Para evitar que los consumidores quedaran bloqueados esperando datos que ya no iban a llegar, el apagado se desacopló mediante un contador protegido por un lock (`active_sensors`) y una señal global (`exit_event`). El último sensor en terminar se encarga de avisar que ya no habrá más lecturas y desbloquea a los módulos de análisis.

Otra decisión importante fue sacar el procesamiento pesado de la zona crítica. La clasificación del tráfico como "CONGESTIONADO" o "Fluido" se hace fuera del bloqueo del mutex. Así se aprovecha mejor el paralelismo y el recurso compartido se retiene el menor tiempo posible.

En cuanto al estilo del código, se siguió la norma PEP 8: las configuraciones del sistema (`BUFFER_SIZE`, `NUM_SENSORS`) van en mayúsculas sostenidas, mientras que los recursos compartidos y los estados mutables (`shared_buffer`, `active_sensors`) usan `snake_case`.

2. Algoritmos y mecanismos de sincronización

El control de concurrencia se apoya en el patrón clásico de productor-consumidor con tres semáforos (`threading.Semaphore`).

El primero es `mutex`, un semáforo binario que arranca en 1. Su función es garantizar exclusión mutua sobre el buffer compartido: solo un hilo puede leer o escribir a la vez.

El segundo es `empty_slots`, un semáforo contador que inicia en `BUFFER_SIZE` (5). Lleva la cuenta de los espacios libres y bloquea a los sensores cuando el buffer está lleno, evitando así un desbordamiento.

El tercero es `full_slots`, también contador, pero que empieza en 0. Indica cuántas lecturas hay disponibles y bloquea a los analizadores cuando el buffer está vacío, lo que evita la espera activa.

El flujo que sigue un sensor (productor) es este: captura la lectura de tráfico, luego hace `empty_slots.acquire()`, después `mutex.acquire()`, inserta la lectura en `shared_buffer`, libera mutex con `mutex.release()` y finalmente hace `full_slots.release()`.

El módulo de análisis (consumidor), por su parte, hace `full_slots.acquire()`, luego `mutex.acquire()`, extrae la lectura de `shared_buffer` (o valida `exit_event`), libera mutex con `mutex.release()`, hace `empty_slots.release()` y por último procesa el estado del flujo vehicular.

Vale la pena señalar que se eliminó por completo la espera activa. Ahora los hilos quedan en reposo gestionados por el sistema operativo hasta que el semáforo correspondiente los despierta. Los registros con marca de tiempo e identificador de hilo dejaron ver una alternancia correcta y un paralelismo real entre los cinco hilos en ejecución.

3. Conclusiones y desempeño en el contexto del SIGET

En todas las pruebas, la ocupación del buffer se mantuvo estrictamente entre 0 y 5 elementos. No hubo pérdidas, corrupción ni duplicación de lecturas de velocidad o conteo de vehículos.

La simulación demuestra que el SIGET puede recibir varios flujos de datos a ritmos asíncronos y procesarlos de forma continua sin saturar el sistema ni dejar hilos colgados. Eso es justamente lo que se espera de una infraestructura crítica.

Además, la arquitectura desacoplada permite aumentar la cantidad de sensores o de módulos de análisis sin tener que tocar la lógica central de sincronización. El patrón escala sin complicaciones.